Same Quantization, Different Failures: Multi-Turn Tool-Calling in Small Language Models

Rayansh Singh
Independent Researcher
rayansh365@gmail.com

Abstract

Post-training quantization is common in local deployments of tool-calling models, but its effect on small language models' multi-turn behavior is not well understood. We evaluate six small open-weight models at four GGUF precisions (F16, Q8_0, Q5_K_M, Q4_K_M) on the multi-turn portion of BFCL v4, parsing executed tool calls into runtime-derived failure categories alongside final task accuracy. Four of the six sit at a full-precision floor or are output-format-limited; our significant results hold for the other two, xLAM-2-3B and Qwen3-1.7B, not the full set. At Q4_K_M their task accuracy falls from 58.00% to 51.75% (xLAM-2-3B) and 14.50% to 8.75% (Qwen3-1.7B), both significant under a paired McNemar test with Bonferroni correction. Yet their failure signatures differ: xLAM-2-3B's per-call success rate is stable while Qwen3-1.7B's wrong-argument rate rises from 20.8% to 26.9%, and Qwen3-1.7B's degradation increasingly takes the form of error-free-but-wrong trajectories that no runtime channel captures. Gemma-3-4B-it is also significant (9 of 400 tasks) but floor-adjacent, hence corroborating rather than headline. Our contribution is a reusable diagnostic: per-call, per-trajectory, and clean-but-failed (silent-failure) rates read together, with a floor/format check that separates a quantization effect from a full-precision ceiling. Aggregate accuracy hides which failure mode grows, and these two models show why that decomposition is needed.

1 Introduction

Open-weight language models in the 1–4B parameter range are increasingly used as tool-calling agents on consumer hardware. In this setting, models are commonly converted to GGUF and served with llama.cpp [6]. Since a model can produce a plausible first tool call yet fail later in an interaction, single-turn function-calling scores do not fully characterize deployment reliability.

The "Flat Score, Amplified Failures" study [1] reports that AWQ/INT4 quantization of 26–35B models can increase per-tool-call failures while leaving aggregate τ^2-bench task success largely unchanged. The authors attribute this pattern to quantization perturbing decisions that were already close to the model's boundary. It is unclear whether this observation extends to smaller models, GGUF quantization, and multi-turn trajectories. Adjacent work leaves this gap open: prior work on compressed agentic capability varies bit budget but evaluates isolated pipeline steps rather than closed-loop trajectories [2]; other work studies GGUF quantization only for single-turn function-call generation [3]; and TinyLLM evaluates small models on BFCL multi-turn but does not vary quantization [4].

Contribution. Our primary deliverable is a diagnostic method, not a single effect size. We decompose quantization degradation into per-call, per-trajectory, and clean-but-failed views. The clean-but-failed view isolates silent failures, trajectories that raise no runtime error yet miss BFCL's

40th Conference on Neural Information Processing Systems (NeurIPS 2026). Workshop: SLMs for Agentic Systems, Paris, France, 2026.

target end state, which no error-channel taxonomy can detect. We pair this with a floor/format protocol that separates a genuine quantization effect from a full-precision capability ceiling (a confound in four of our six models). This toolkit is designed to transfer across models, quantizers, and benchmarks, and the study below is its first application, one we are actively scaling (§4). Applied to a regime that [1] did not test (1–4B models, GGUF rather than AWQ/GPTQ, BFCL multi-turn), it already surfaces a non-uniform result. xLAM-2-3B reproduces the flat-call/lower-task-accuracy dissociation of [1] (which we do not re-claim), with degradation that tracks wrong-call-order and state-mismatch rather than wrong-argument calls, though only a minority of its degraded tasks fail that way (§4), so we hold it as a hypothesis, not a mechanism. Qwen3-1.7B instead degrades directly through wrong-argument failures at both granularities. That two similarly-sized models under an identical serving stack show different failure signatures under the same quantization step is our headline empirical observation, a case study on two models, not a general law.

2 Method

Models. We evaluate six models registered in the Berkeley Function-Calling Leaderboard (BFCL): Llama-3.2-1B-Instruct, Llama-3.2-3B-Instruct, Qwen3-1.7B, xLAM-2-3B-fc-r, Phi-4-mini-instruct, and Gemma-3-4B-it. We focus on models between 1B and 4B parameters because lower-capability models are more likely to have floor-level full-precision performance [4]. Five use BFCL's function-calling handlers (a dedicated fine-tune emits tool-call syntax directly) whereas Gemma-3-4B-it is registered only through a prompt-based handler that inserts function definitions into the prompt and parses the free-form completion. Gemma is numerically unstable under fp16, so we use BF16 as its reference arm. Since its reference accuracy is already low, its BF16-to-Q4_K_M change reflects only 9 of 400 tasks flipping (Table 4), so we weight its trend more cautiously than xLAM-2-3B's or Qwen3-1.7B's (§3).

Quantization. All arms, including the F16 reference, use the same llama.cpp/`llama-server` path: HF checkpoint → GGUF F16 → GGUF Q8_0, Q5_K_M, or Q4_K_M via `llama-quantize`. Holding the serving path fixed avoids conflating quantization with differences in inference engines, sampling, or positional encoding. The main K-quant arms (Q5_K_M, Q4_K_M) use no imatrix calibration, which isolates the quantization step but is a lower-quality target than the calibrated K-quants practitioners typically deploy; we report an imatrix-calibrated Q4_K_M for the two headline models as a deployment-representative comparison in §3 (Table 2). `llama-server` runs with `-ngl -1` (full GPU offload), `-c 16384` context, and `-np 1` (serializing requests rather than batching, avoiding batch-size-dependent kernel reduction-order effects). All GPU arms run on Kaggle T4x2 sessions (`machine_shape` pinned to `NvidiaTeslaT4` in every kernel's metadata), so hardware is held constant across arms.

Decoding. We run `bfcl generate` at its default temperature (0.001, effectively greedy), with no explicit seed. Floating-point non-determinism in quantized CUDA kernels (e.g., reduction-order effects) can still perturb outputs, but its effect under greedy decoding is bounded and we measure it directly with a repeated-trial check (§4).

Benchmark and metrics. We use BFCL v4's multi-turn subset: `multi_turn_base`, `multi_turn_miss_func`, `multi_turn_miss_param`, and `multi_turn_long_context`, with 100 tasks per category. BFCL evaluates the final system state rather than a live, simulated user. We report task accuracy and per-call outcomes. BFCL's `error_type` field provides one coarse tag for a failed task. We additionally parse the raw trajectories and use the tool runtime message to categorize each executed call as success, malformed output, wrong argument, wrong call order, state mismatch, or over/under-calling. This categorization is a regex-based classifier over the tool's runtime error string (§4 details its limits). To check its accuracy, we hand-labeled a random sample of 45 failed calls, drawn across both headline models and all four precisions, and compared our label to the classifier's automatic channel. The two agreed on 43/45 (95.6%). Per-channel precision on this sample is 27/28 (96%) for wrong-argument, 9/9 for wrong-call-order, 5/5 for malformed-output, and 2/2 for state-mismatch; the two channels the xLAM-2-3B account depends on (wrong-call-order and state-mismatch, §3) thus show no false positives here, though on small counts. Both disagreements were adjacent-precondition cases in which a generic "already exists"/"no such file" runtime string was auto-labeled wrong-argument or over/under-calling but, read against the surrounding call se-

Table 1: Multi-turn task accuracy (mean over four BFCL categories, n=100 per category) by GGUF precision. †Gemma-3-4B-it's reference/highest-precision arm is BF16, not F16 (§2).

Model	F16	Q8_0	Q5_K_M	Q4_K_M
xLAM-2-3B	58.00%	56.00%	56.00%	**51.75%**
Qwen3-1.7B	14.50%	13.25%	14.75%	**8.75%**
Llama-3.2-3B	5.00%	5.25%	3.50%	4.25%
Gemma-3-4B-it	2.25%†	1.50%	0.50%	**0.00%**
Llama-3.2-1B	0.00%	0.00%	0.00%	0.00%
Phi-4-mini	0.00%	0.00%	0.00%	0.00%

quence, more plausibly reflects a missing earlier step (wrong call order); both were corrected toward wrong-call-order by the human label. A larger, multi-annotator validation set is left to future work.

Statistical testing. BFCL evaluates the same 400 tasks at every arm, so a precision-vs-precision difference is paired. Recovering the per-task pairing from the raw logs, we report McNemar's exact test on the discordant pairs and a 95% CI from a category-stratified paired bootstrap (10,000 resamples). We treat the six reference-vs-Q4_K_M comparisons as the primary family and Bonferroni-correct across them ($\alpha \approx 0.0083$); §3 also reports a stricter correction across all 18 model×precision comparisons.

3 Results

Task accuracy. Table 1 shows a clear Q4_K_M decline for xLAM-2-3B (a 6.25pp drop) and Qwen3-1.7B (5.75pp). Qwen3-1.7B's Q5_K_M score slightly exceeds F16, so its evidence is strongest for an effect at the lowest precision rather than a monotonic decline across all four formats; xLAM-2-3B holds flat through Q5_K_M and drops only at Q4_K_M. BFCL evaluates the same 400 tasks at every arm, so Table 4 tests each model's reference-vs-Q4_K_M change directly as a paired binary outcome (§2). Both remain significant after Bonferroni correction across the six models, and under the stricter correction across all 18 model×precision comparisons ($\alpha \approx 0.0028$), with bootstrap 95% CIs excluding zero; Llama-3.2-3B does not, consistent with its quantization signal being noise relative to a genuine multi-turn state-tracking weakness (§4). The discordant pairs also skew one way: both headline models' F16-pass/Q4_K_M-fail count is more than double the reverse (Table 4), the systematic imbalance McNemar's test detects, rather than the roughly even split symmetric sampling noise would produce. Gemma-3-4B-it is the only model declining monotonically across all four arms. Its change survives the 6-model correction but not a stricter 18-comparison one ($\alpha \approx 0.0028$), and its 2.25% baseline places it in the floor-adjacent band, so we treat it as a corroborating rather than headline effect (§2). Cohen's h is small for all three models with a significant McNemar p but ranks Gemma's highest, an arcsine-transform artifact near the ends of $[0, 1]$ (its drop is proportionally large against a 2.25% baseline). We therefore read the discordant-pair counts and McNemar p-values (Table 4) as the more informative summary.

Calibrated Q4_K_M. As noted in §2, the main K-quant arms use no imatrix calibration. Table 2 adds an imatrix-calibrated (wikitext-2) Q4_K_M for the two headline models as the more deployment-representative operating point. Calibration recovers 2.00pp of xLAM-2-3B's 6.25pp drop (32%) and 4.00pp of Qwen3-1.7B's 5.75pp drop (70%): the degradation shrinks but does not vanish. We read the calibrated residual as the effect a deployer should expect and the uncalibrated drop (Table 4) as a mechanism-isolating upper bound. The calibrated arm now has three-trial and paired-test support (cross-trial stdev 0.00 and 0.38pp): McNemar on F16 vs. calibrated Q4_K_M keeps xLAM-2-3B significant (p=0.04) but not Qwen3-1.7B (p=0.41), so calibration erases the general model's uncalibrated-significant drop (Table 4) but not the specialist's.

Does the signature persist under calibration? Table 2 reports only accuracy for the calibrated arm; Table 3 repeats the per-call/per-trajectory decomposition (§2) at that arm to check whether each model's failure signature survives calibration, not just its effect size. For Qwen3-1.7B, calibration recovers the signature along with the accuracy: per-call wrong-argument returns to within 1.1pp of F16 (20.8% → 21.9%, vs. 26.9% uncalibrated, Table 5), and the clean-but-failed rise loses

Table 2: Uncalibrated vs. imatrix-calibrated (wikitext-2) Q4_K_M multi-turn task accuracy for the two headline models. Calibration shrinks the F16-to-Q4_K_M drop but does not remove it; the calibrated residual is significant for xLAM-2-3B but not Qwen3-1.7B (paired McNemar).

Model	F16	Q4_K_M	Q4_K_M + imat	Residual vs. F16
xLAM-2-3B	58.00%	51.75%	53.75%	−4.25pp
Qwen3-1.7B	14.50%	8.75%	12.75%	−1.75pp

Table 3: Does a failure signature persist under imatrix calibration? Each cell is a paired McNemar test on a binary trajectory flag, F16 vs. the named arm (n=361–379; §2); Δ is the F16-to-arm change in % of trajectories with that flag. Bold: $p < 0.05$.

Model	Signature (trajectory-level)	Uncalibrated Δ (p)	Calibrated Δ (p)
Qwen3-1.7B	clean-but-failed	**+8.2pp (0.0003)**	+1.7pp (0.556)
xLAM-2-3B	wrong-order ∪ state-mismatch	**+3.4pp (0.024)**	−2.9**pp (0.027)**
xLAM-2-3B	clean-but-failed	+2.1pp (0.312)	**+4.5pp (0.014)**

significance. For xLAM-2-3B, calibration does not merely shrink the uncalibrated signature, it reverses it: wrong-order∪state-mismatch falls significantly *below* F16, while clean-but-failed rises further and now significantly. xLAM-2-3B's calibrated accuracy residual (Table 2) is thus carried by silent failures, not the visible call-order/state channel §4 built its hypothesis on; we revisit that reading there. This also sharpens the upper-bound framing above: for xLAM-2-3B, calibration overturns not just the size of the uncalibrated signature but its direction.

Sensitivity of the clean-but-failed result to excluded trajectories. Table 6's denominators exclude trajectories with an inference-endpoint error and no recorded call; that exclusion rate is flat at 5.2–5.5% across all five xLAM-2-3B arms (including calibrated) but rises from 5.8% (F16) to 9.0% (calibrated Q4_K_M) for Qwen3-1.7B. Under the most conservative assignment of these excluded tasks (treating none as clean-but-failed, the bias that most shrinks the effect), Qwen3-1.7B's uncalibrated clean-but-failed rate over the full 400-task denominator still rises 14.5%→22.2% (+7.7pp), close to the paired estimate (+8.2pp above); the opposite (most generous) assignment gives +16.3pp. The same bound on xLAM-2-3B's calibrated clean-but-failed rise gives +4.2pp, close to its paired estimate (+4.5pp, Table 3). Both silent-failure results hold under either bound.

Per-category breakdown. The two declines also fall in different categories (Appendix A, Table 8): xLAM-2-3B's is largest in `base` (−10pp) and `long_context` (−8pp), Qwen3-1.7B's largest in `miss_func` (−12pp) and `miss_param` (−8pp). This split is what the training-based reading in §4 predicts, since the two category pairs stress cross-turn sequencing and argument construction respectively. The categories were not designed to isolate that interaction, so we treat the alignment as corroborating rather than decisive.

Per-call outcomes. Table 5 summarizes 2,700–4,400 executed calls per arm. For xLAM-2-3B the per-call success rate stays flat (83.5–85.0%) even as task accuracy falls. The flat call rate matters because task accuracy is falling underneath it: uniform call corruption would raise the aggregate error rate, so the drop must instead come from a small set of already-marginal trajectories flipping across BFCL's all-or-nothing boundary. This is [1]'s near-decision-boundary mechanism: the call-level metric stays flat while the task-level effect goes undetected. Qwen3-1.7B's call success is also near-flat, but differently: wrong-argument rises (20.8% to 26.9%) while malformed-output falls, canceling in the pooled rate. The aggregate call view understates both models (concentration for xLAM-2-3B, cancellation for Qwen3-1.7B), so we decompose by trajectory next.

Trajectory-level outcomes. Table 5 pools every executed call into one denominator, which can hide a channel that shifts just enough to flip individual trajectories without moving the aggregate call rate. Table 6 instead asks, per trajectory: does it contain at least one call in a given channel? For xLAM-2-3B, the trajectory-level wrong-argument rate is essentially flat, mirroring its flat per-call rate, while wrong-call-order and state-mismatch both rise, a more precise account of which failure types actually move than the call-level view alone. For Qwen3-1.7B, the wrong-argument trajectory rate

Table 4: Paired significance of the reference→Q4_K_M accuracy change, n=400 tasks per model (McNemar's exact test on discordant task pairs; 95% CI from a category-stratified paired bootstrap, 10,000 resamples). Cohen's h is the arcsine-transformed effect size between the reference and Q4_K_M accuracies (conventional thresholds: 0.2 small, 0.5 medium, 0.8 large). †Gemma-3-4B-it's reference arm is BF16 (§2). Bold: significant at $\alpha = 0.05/6 \approx 0.0083$ (Bonferroni across the six models compared at Q4_K_M).

Model	Diff (pp)	95% CI (pp)	Discordant (b, c)	McNemar p	Cohen's h
xLAM-2-3B	-6.25	$[-10.00, -2.50]$	$(45, 20)$	**0.0026**	-0.126
Qwen3-1.7B	-5.75	$[-9.50, -2.25]$	$(39, 16)$	**0.0027**	-0.181
Gemma-3-4B-it†	-2.25	$[-3.75, -1.00]$	$(9, 0)$	**0.0039**	-0.301
Llama-3.2-3B	-0.75	$[-2.50, 0.75]$	$(7, 4)$	0.549	-0.036
Llama-3.2-1B	0.00	$[0.00, 0.00]$	$(0, 0)$	1.000	0.000
Phi-4-mini	0.00	$[0.00, 0.00]$	$(0, 0)$	1.000	0.000

Table 5: Per-call outcome rates for xLAM-2-3B and Qwen3-1.7B, aggregated across categories. The omitted other category is at most 0.4% in every arm.

Model	Precision	Success	Malformed	Wrong-arg	Wrong-order	State-mismatch
xLAM-2-3B	F16	83.5%	1.8%	10.7%	2.8%	0.6%
	Q8_0	83.8%	2.1%	10.1%	2.8%	0.5%
	Q5_K_M	84.3%	2.0%	8.1%	3.0%	2.0%
	Q4_K_M	85.0%	1.6%	8.7%	3.3%	0.7%
Qwen3-1.7B	F16	64.2%	8.0%	20.8%	3.8%	1.2%
	Q8_0	61.8%	7.4%	22.4%	4.6%	1.2%
	Q5_K_M	60.4%	10.6%	22.4%	4.5%	0.5%
	Q4_K_M	62.6%	3.1%	26.9%	4.3%	1.2%

rises and malformed-output falls from F16 to Q4_K_M (despite a non-monotonic Q5_K_M spike to 51.1%, Table 6), corroborating both per-call directions at the endpoints (Table 5). But Qwen's Clean rate (every call succeeds) counter-intuitively rises from 23.1% to 32.0% even as task accuracy falls. Joining cleanliness to BFCL pass/fail resolves the paradox: the extra clean trajectories are almost all failures. Clean-and-passed trajectories are unchanged (29 at both arms), while clean-but-failed ones rise from 58 to 89, or 15.4% to 24.1% of scored trajectories. A paired McNemar test (n=366) confirms the rise is not noise: $+8.2$pp (95% CI $[+4.1, +12.6]$pp, p=0.0003). The paired estimate is slightly below the marginal-rate difference because it is restricted to the 366 tasks scored at both arms, whereas the two marginal rates use each arm's own denominator (infra errors excluded per arm). That is, a growing share of Qwen's Q4_K_M failures are trajectories that execute without a single runtime call error yet miss BFCL's target end state, failures no error-string channel can tag (an omitted required call, or a call that succeeds without reaching the goal), visible only as the widening Clean-vs-accuracy gap. xLAM-2-3B's clean-but-failed share stays stable (14.2% to 16.4%), consistent with its drop being carried by visible wrong-call-order/state-mismatch channels, not silent failures. The Clean rate must thus be read against task accuracy: Qwen's fewer malformed-output failures did not become correct trajectories, but error-free-but-wrong ones.

Floor and format effects. Llama-3.2-1B and Phi-4-mini score 0% at every precision including F16, so their results cannot isolate a quantization effect; Table 4 confirms zero discordant pairs for both. Llama-3.2-3B's nonzero change in Table 1 is likewise not significant ($p = 0.549$). To distinguish a capability floor from an output-format problem, we apply a lenient, turn-0-only diagnostic that checks whether the completion names the correct function regardless of wrapper syntax (Table 7).

Llama-3.2-1B rarely attempts BFCL's bracket syntax (0.9% of completions; 64.7% free prose) and its lenient rate is flat within noise (7.9–11.1%), a capability rather than formatting limitation. Phi-4-mini often identifies the right function but wraps it in prose instead of its documented `<|tool_call|>` tag (token/prompt/BOS all matched the source config); its lenient rate is non-monotonic (29.4–43.0%), read as heuristic noise. Gemma-3-4B-it's lenient rate declines overall from BF16 to Q4_K_M (47.2% to 35.4%, with a small non-monotonic uptick at Q8_0), matching its strict-accuracy direction, the most credible of the three floor-adjacent signals, though a name-only proxy. Llama-3.2-3B is near the

Table 6: Trajectory-level outcome rates for xLAM-2-3B and Qwen3-1.7B: the fraction of trajectories containing at least one call in each channel (Clean = every call in the trajectory succeeded), aggregated across categories. Rows need not sum to 100% (unlike Table 5) since one trajectory can contain calls from more than one channel. Denominators exclude the small number of trajectories with an inference-endpoint error and no recorded call attempt (0.3–7.8% per arm), matching Table 5's exclusion of infra errors. Omitted channels are small and flat across arms: over/under-calling is 4.0–4.9%, hallucinated-tool and missing-argument are 0.0%, and other is $\leq$1.1% (xLAM-2-3B) / $\leq$3.5% (Qwen3-1.7B).

Model	Precision	Clean	Malformed	Wrong-arg	Wrong-order	State-mismatch
xLAM-2-3B	F16	50.4%	13.2%	25.9%	13.7%	3.4%
	Q8_0	48.8%	15.0%	25.6%	13.5%	2.9%
	Q5_K_M	48.7%	14.3%	25.4%	14.6%	3.2%
	Q4_K_M	48.3%	11.6%	25.1%	16.4%	5.0%
Qwen3-1.7B	F16	23.1%	42.4%	37.1%	17.8%	4.5%
	Q8_0	28.4%	42.1%	30.6%	17.7%	5.1%
	Q5_K_M	22.6%	51.1%	36.7%	17.6%	3.2%
	Q4_K_M	32.0%	28.7%	40.9%	16.0%	6.0%

Table 7: Lenient (name-only, turn-0) content-correctness rate. The three floor-adjacent models are shown against the two headline models as a format-competence reference band; compare each to its strict BFCL task accuracy (Table 1). †Gemma-3-4B-it's reference arm is BF16.

Model	F16/BF16	Q8_0	Q5_K_M	Q4_K_M
Headline models (usable operating point):				
xLAM-2-3B	77.5%	76.3%	76.3%	75.9%
Qwen3-1.7B	59.5%	58.9%	55.1%	60.8%
Floor-adjacent models:				
Llama-3.2-1B	11.1%	8.9%	7.9%	10.4%
Phi-4-mini	31.0%	29.4%	32.0%	43.0%
Gemma-3-4B-it†	47.2%	48.4%	42.7%	35.4%

same floor (3.5–5.25%) despite correct formatting, pointing to a multi-turn state-tracking limitation. We therefore do not read a quantization trend from Llama-3.2-1B or Phi-4-mini, and treat Gemma-3-4B-it's as suggestive. By the same diagnostic, the two headline models are not format-limited: their lenient turn-0 rates (Table 7) are far above strict multi-turn accuracy and flat across precision. For Qwen3-1.7B, whose 14.50% baseline might otherwise invite grouping with the floor models, this confirms the low strict score reflects genuine multi-turn difficulty (it names the correct turn-0 function well over half the time), not the formatting failure pinning Llama-3.2-1B and Phi-4-mini to zero. Its Q4_K_M effect is thus a multi-turn execution effect, not an artifact of a near-floor baseline.

4 Discussion and Limitations

For xLAM-2-3B, the Q4_K_M drop tracks with rising wrong-call-order and state-mismatch rates (Table 6), not the flat wrong-argument rate, but this is the dominant direction of change, not the mechanism behind most degraded tasks. Of 45 F16-pass/Q4_K_M-fail tasks, only 6 fail via such a call (§B), the rest through mixed channels or the silent, error-free-but-wrong route quantified for Qwen3-1.7B in §3. A paired test confirms this rise is real but modest (n=379, +3.4pp, p=0.02), an order of magnitude weaker than Qwen3-1.7B's shift above, and still not the majority mechanism behind individual failures. The association is thus suggestive, not causal, though the labels behind it are reliable (95.6% agreement, §2). Qwen3-1.7B instead shifts toward wrong-argument calls and trajectories: the same quantization step, a qualitatively different signature. This xLAM-2-3B signature is also specific to uncalibrated quantization: under imatrix calibration the same wrong-order$\cup$state-mismatch rate falls significantly *below* F16 ($-$2.9pp, p=0.03, Table 3) even as xLAM-2-3B's calibrated accuracy residual stays significant (Table 2) and its clean-but-failed rate rises instead (+4.5pp, p=0.01). The visible call-order/state-mismatch channel is therefore not the mechanism

behind xLAM-2-3B's calibrated, deployment-realistic degradation; whatever degrades it under calibration is itself silent, the same failure mode Qwen3-1.7B's uncalibrated drop runs through.

We propose one hypothesis, for the uncalibrated signature specifically: quantization noise surfaces first in whichever competence a model has drilled least. xLAM-2-3B is a dedicated function-calling fine-tune, so single-call argument construction is over-rehearsed and robust, and the noise instead perturbs the softer cross-turn sequencing. Qwen3-1.7B is a general instruct model whose argument construction is itself the least-hardened link and degrades first. The hypothesis predicts where each model should fail, and the per-category split (§3) is consistent: Qwen3-1.7B's loss concentrates in the miss-func/miss-param categories that most stress argument construction, xLAM-2-3B's in the base/long-context categories that stress sequencing. It is also falsifiable: the same FC fine-tuning applied to a comparable general model should move its degradation from argument toward sequencing errors, a test our fixed model set cannot run, so we flag it as the reading's central untested prediction, not a result. It also describes the uncalibrated mechanism specifically: xLAM-2-3B's visible sequencing signature does not survive calibration (above), so this hypothesis explains why uncalibrated Q4_K_M fails each model differently, not what happens at the calibrated, deployment-realistic operating point.

Practical significance. Effects are statistically robust but small ($|h| < 0.19$ for the two headline models; smaller still calibrated, Table 2); we do not claim quantization is catastrophic for small tool-callers. That small aggregate effect is why the decomposition earns its keep: the call-, trajectory-, and clean-but-failed views reveal which failure mode grows even when the headline barely moves.

Limitations: for the two headline models, three decoding trials per arm give cross-trial accuracy stdev of 0.00–0.25pp (xLAM-2-3B) and 0.18–1.03pp (Qwen3-1.7B), 5–25$\times$ smaller than the $\sim$6pp drop, leaving thread-level CUDA non-determinism as the residual source (other four models untested). Per-call rates (Table 5) treat calls as independent though nested within trajectories, so we read them descriptively. Our models sit in the 1.7–4B range; we do not evaluate above 4B, below 1B, or Qwen3-4B-Instruct-2507, and BFCL's state-based scoring cannot capture adversarial user interactions. The full analysis pipeline and the aggregated tables behind every result are anonymized supplementary material with a README mapping tables to code; raw per-task transcripts will be released upon publication.

Future work. A wider model range (Qwen3-4B-Instruct-2507, sub-1B); perturbations testing the xLAM-2-3B wrong-call-order/state-mismatch hypothesis; a fine-tuned Gemma-3-4B-it handler for its confound; and a simulated-user (τ^2-bench-style) setting for adversarial turns.

Conclusion. On BFCL multi-turn, Q4_K_M significantly lowers paired task accuracy for xLAM-2-3B and Qwen3-1.7B (McNemar, $p < 0.01$; 2 of 6 models, the rest floor- or format-limited), but through different failure signatures. Assessing a quantized small tool-caller therefore calls for the call-, trajectory-, and clean-but-failed decomposition we release here, not aggregate accuracy alone.

Acknowledgments and Disclosure of Funding

References

[1] Jiwon Jang, Kisu Yang, Heuiseok Lim, and Hyunwoo Park. Flat Score, Amplified Failures: How the Error Budget Masks Damage in Quantized LLM Agents. *arXiv preprint arXiv:2607.27275*, 2026.

[2] Peijie Dong, Zhenheng Tang, Xiang Liu, Lujun Li, Xiaowen Chu, and Bo Li. Can Compressed LLMs Truly Act? An Empirical Evaluation of Agentic Capabilities in LLM Compression. *arXiv preprint arXiv:2505.19433*, 2025.

[3] Ishan Kavathekar, Raghav Donakanti, Ponnurangam Kumaraguru, and Karthik Vaidhyanathan. Small Models, Big Tasks: An Exploratory Empirical Study on Small Language Models for Function Calling. *arXiv preprint arXiv:2504.19277*, 2025.

[4] Mohd Ariful Haque, Fahad Rahman, Kishor Datta Gupta, Khalil Shujaee, and Roy George. TinyLLM: Evaluation and Optimization of Small Language Models for Agentic Tasks on Edge Devices. *arXiv preprint arXiv:2511.22138*, 2025.

[5] Shishir G. Patil, Huanzhi Mao, Charlie Cheng-Jie Ji, Fanjia Yan, Vishnu Suresh, Ion Stoica, and Joseph E. Gonzalez. The Berkeley Function Calling Leaderboard. *ShishirPatil/gorilla*, GitHub, 2024.

[6] Georgi Gerganov et al. llama.cpp: LLM Inference in C/C++. *ggml-org/llama.cpp*, GitHub, 2023.

A Per-category accuracy

Table 8 unpacks the four-category mean of Table 1 for the two headline models, supporting the per-category discussion in §3.

Table 8: Per-category task accuracy for xLAM-2-3B and Qwen3-1.7B (n=100 per category), unpacking the four-category mean in Table 1.

Model	Precision	Base	Miss-func	Miss-param	Long-context
xLAM-2-3B	F16	72%	53%	55%	52%
	Q8_0	69%	52%	56%	47%
	Q5_K_M	72%	52%	55%	45%
	Q4_K_M	62%	48%	53%	44%
Qwen3-1.7B	F16	14%	19%	13%	12%
	Q8_0	15%	17%	10%	11%
	Q5_K_M	20%	16%	16%	7%
	Q4_K_M	16%	7%	5%	7%

B Worked xLAM-2-3B trajectories

Table 6 reports rates; this appendix shows four concrete xLAM-2-3B tasks that pass at F16 and fail at Q4_K_M via wrong-call-order or state-mismatch, to make the association in §4 concrete rather than purely statistical. All four pass at F16 and every non-reference precision above Q4_K_M; Q4_K_M is the only arm where each fails.

`multi_turn_base_56` **(state mismatch).** The user asks, across three turns, to check a distance, then fill the tank to end with 50 gallons, then start the engine. At Q4_K_M, the model calls `fillFuelTank(fuelAmount=50.0)` directly rather than first checking the tank's remaining capacity; the environment returns "Cannot fill gas above the tank capacity." The requested absolute end-amount (50 gallons) exceeds what the tank can hold from its current level, so satisfying the instruction literally requires checking capacity first, a check the model skips only at Q4_K_M.

`multi_turn_miss_func_95` **(wrong call order, cascading).** The user states doors are secured and the parking brake is engaged, then asks to start the engine. At Q4_K_M, `startEngine` is called immediately and fails ("Brake pedal needs to be pressed when starting the engine"), despite the user's stated setup implying the precondition should already hold. Three turns later, a distance lookup (`estimate_distance`) is retried five times with different destination-city arguments, each failing with "distance not found in database", consistent with the model probing for a valid value after the early failure rather than the two errors being independent.

`multi_turn_miss_param_128` **(state mismatch).** The user asks to buy 150 shares of a Technology-sector stock at market price, later clarified as Microsoft. At Q4_K_M, `place_order` is called with `price=2840.34`, the going rate for a different stock quoted earlier in the conversation (Alphabet/GOOG) rather than Microsoft's, and the environment rejects it: "Insufficient funds: required $426051.00 but only $55000.00 available." The failure surfaces as a resource-precondition (state) error, but its root cause traces back to a wrong argument value carried over from an earlier turn.

`multi_turn_long_context_51` **(wrong call order, two independent instances).** The user asks the model to verify doors are locked, then start the engine, then check tire pressure, then send a message to a contact. At Q4_K_M, `startEngine` fails on the same brake-pedal precondition as above, and separately, the later `send_message` call fails with "No user is currently logged in", a second, unrelated precondition (authentication) that the model also does not satisfy first. Two independent call-order failures in one trajectory illustrate why the trajectory-level wrong-call-order rate (Table 6) can rise without a proportional rise in the per-call rate (Table 5): a single trajectory can contain more than one such failure without that concentration showing up in a call-pooled rate.

Of the 45 xLAM-2-3B tasks that pass at F16 and fail at Q4_K_M, 6 fail via a wrong-call-order or state-mismatch call (1 wrong-call-order only, 1 wrong-call-order plus a wrong-argument call, 3 state-mismatch only, 1 state-mismatch plus a wrong-argument call); these four are a convenience sample of those 6, selected to be readable, not a random sample (§2 describes the random sample used for taxonomy validation). Read together, though, they share a pattern: each failure is a skipped or stale precondition check (tank capacity, brake-before-start, stock price carried from an earlier turn, login-before-messaging) rather than a garbled argument value, which is consistent with §4's reading that Q4_K_M perturbs xLAM-2-3B's state-tracking across turns rather than its argument construction within a single call.